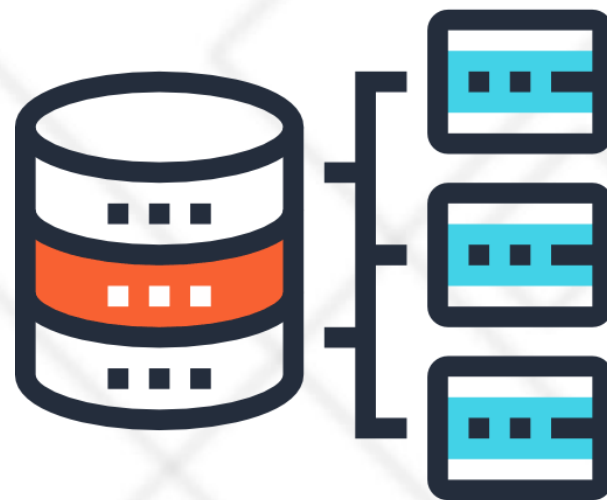




北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY



第8章 数据库恢复技术

北京理工大学 计算机学院

段春晖

duanch@bit.edu.cn

第8章 数据库恢复技术



- 8.1 事务的基本概念和特征
- 8.2 数据库恢复的必要性
- 8.4 数据转储与恢复
- 8.5 基于日志的数据库恢复
- 8.3 数据库恢复策略
- 8.6 检查点恢复技术
- 8.7 数据库镜像恢复技术
- 8.8 SQL Server的数据恢复机制
- 8.9 小结

8.1 事务的基本概念和特征

- 事务 (Transaction) 是用户定义的一个数据库操作序列，这些操作要么全做，要么全不做，是一个不可分割的工作单位
- 事务是一系列的数据库操作，事务是恢复和并发控制的基本单位
- 事务和程序是两个概念
 - 在关系数据库中，一个事务可以是一条SQL语句，一组SQL语句或整个程序
 - 一个应用程序通常包含多个事务
- 事务处理技术主要包括数据库恢复技术和并发控制技术

8.1.1 事务的基本概念

□如何定义事务

- 显式定义方式

BEGIN TRANSACTION

SQL 语句1

COMMIT

BEGIN TRANSACTION

SQL 语句1

ROLLBACK

- 隐式方式：当用户没有显式地定义事务时，DBMS按缺省规定自动划分事务

□COMMIT 提交事务

- 事务正常结束，提交事务的所有操作（读+更新），**事务中所有对数据库的更新永久生效**

□ROLLBACK 回滚事务

- 事务异常终止
 - ◆ 事务运行的过程中发生了故障，不能继续执行
- 回滚事务的**所有更新操作**，恢复到事务开始时的状态

8.1.2 事务特征

□事务的四个特性：

- 原子性 (Atomicity)、一致性 (Consistency)、隔离性 (Isolation)、持续性 (Durability)，简称**ACID特性**

□1. 原子性 (Atomicity)

- 事务是数据库的逻辑工作单位，事务中包括的诸操作要么都做，要么都不做

□2. 一致性 (Consistency)

- 事务执行的结果必须是使数据库从一个一致性状态变到另一个一致性状态
- 一致性状态：数据库中只包含成功事务提交的结果
- 不一致状态：数据库中包含失败事务的结果

8.1.2 事务特征

银行转帐：从帐号A中取出一万元，存入帐号B。

- 定义一个事务，该事务包括两个操作

A	B
$A=A-1$	$B=B+1$

- 这两个操作要么全做，要么全不做
 - ◆ 全做或者全不做，数据库都处于一致性状态
- 如果只做一个操作，数据库就处于不一致性状态

8.1.2 事务特征

□3. 隔离性

- 对并发执行而言，一个事务的执行不能被其他事务干扰

- ◆ 一个事务内部的操作及使用的数据对其他并发事务是隔离的
- ◆ 并发执行的各个事务之间不能互相干扰

T1的修改被T2覆盖了!

T ₁	T ₂
① 读A=16	读A=16
②	
③ A←A-1 写回A=15	
④	A←A-3 写回A=13

8.1.2 事务特征

□4. 持续性

- 持续性也称永久性 (Permanence)
- 一个事务一旦提交，它对数据库中数据的改变就应该是永久性的
- 接下来的其他操作或故障不应该对其执行结果有任何影响

8.1.2 事务特征

□保证事务ACID特性是事务处理的任务

□破坏事务ACID特性的因素

- 多个事务并行运行时，不同事务的操作交叉执行
 - ◆ 保证原子性
 - ◆ 事务不能相互影响
- 事务在运行过程中被强行停止
 - ◆ 不影响数据库的完整性、一致性
 - ◆ 不影响其它事务

8.1.3 事务状态

- 由于系统故障等原因事务可能无法成功执行而处在中止状态，称为**事务中止 (aborted)**
 - 为了保证事务的原子性，中止事务不能破坏数据库的一致性状态
 - 一个被中止的事务所做过任何改变操作要取消
- 数据库恢复到事务执行前的状态，称之为**事务回滚 (rolled back)**

8.1.3 事务状态

□事务应该处在下列状态之一：

- 活动状态 (active)

- ◆ 初始状态，事务执行时事务处于活动状态

- 部分提交状态 (partially committed)

- ◆ 事务最后一条语句被执行完毕后进入部分提交状态

- ◆ 事务中对数据的操作已经全部完成，但结果数据还驻留在内存中

- ◆ 如果在此状态时，系统出现故障仍可能使事务不得不终止

- 失败状态 (failed)

- ◆ 如果事务不能正常执行，事务就进入失败状态

- ◆ 这意味着事务没有成功地完成，必须回滚

- ◆ 回滚 (Rollback) 就是撤销事务已经做出的任何数据更改

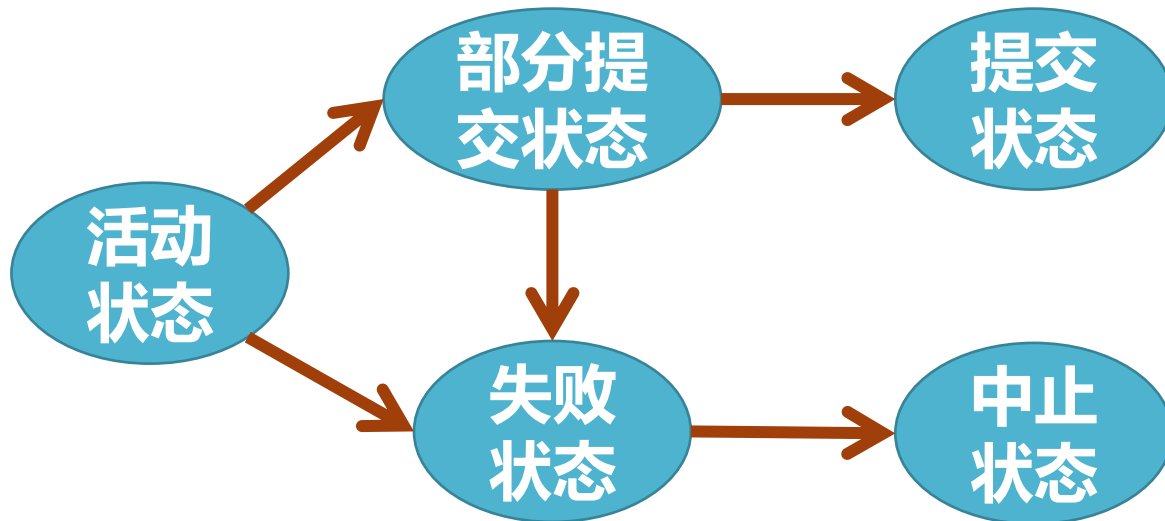
8.1.3 事务状态

□事务应该处在下列状态之一：(续)

- 活动状态 (active)
- 部分提交状态 (partially committed)
- 失败状态 (failed)
- 中止状态 (aborted)
 - ◆ 事务回滚并且数据库已经恢复到事务执行前的状态
- 提交状态 (committed)
 - ◆ 当数据库系统将事务中对数据的更改完全写入磁盘时，写入一条事务日志信息，标志着事务成功完成，这时事务就进入了提交状态

8.1.3 事务状态

- 事务从活动状态开始，当完成最后一条语句后进入部分提交状态
- 此刻，事务虽已完成，但结果数据仍驻留在内存中，某些故障可能导致其失败
- 把结果数据写入外部存储器中，事务进入提交状态
- 当事务不能正常执行时，进入失败状态
- 失败状态的事务必须撤销，事务进入中止状态



第8章 数据库恢复技术



- 8.1 事务的基本概念和特征
- 8.2 数据库恢复的必要性
- 8.4 数据转储与恢复
- 8.5 基于日志的数据库恢复
- 8.3 数据库恢复策略
- 8.6 检查点恢复技术
- 8.7 数据库镜像恢复技术
- 8.8 SQL Server的数据恢复机制
- 8.9 小结

8.2 数据库恢复的必要性

□故障是不可避免的

- 计算机硬件故障、系统软件和应用软件的错误、操作员的失误、恶意的破坏
- 故障的影响
 - ◆ 运行事务非正常中断、破坏数据库

□数据库管理系统对故障的对策

- DBMS提供恢复子系统
- 保证故障发生后，能把数据库中的数据从错误状态恢复到某种逻辑一致的状态
- 保证事务ACID

□恢复技术是衡量系统优劣的重要指标

8.2 数据库恢复的必要性

- 数据库系统可能发生各种各样的故障，大致可分为：
 - 系统故障
 - 事务内部的故障
 - 存储设备故障
 - 其它原因

□什么是系统故障

- 造成系统停止运转的任何事件，使得系统要重新启动
- 整个系统的正常运行突然被破坏、所有正在运行的事务都非正常终止、内存中数据库缓冲区的信息全部丢失、外部存储设备上的数据未受影响

□系统故障的常见原因

- 操作系统或DBMS代码错误
- 操作员操作失误
- 特定类型的硬件错误（如CPU故障）
- 突然停电

事务内部的故障

□什么是事务故障

- 某个事务在运行过程中由于种种原因未运行至正常终止点就夭折了

□事务内部的故障

- 有的是可以通过事务程序本身发现的
 - ◆ 转账事务的例子
- 有的是非预期的

事务内部的故障

□ 银行转账事务，把一笔金额从账户A转给账户B

BEGIN TRANSACTION

读账户A的余额BALANCE;

BALANCE=BALANCE-AMOUNT;

写回BALANCE;

IF (BALANCE < 0) THEN

{

打印“金额不足，不能转账”;

ROLLBACK; (撤销刚才的修改，恢复事务)

}

ELSE

{

读账户B的余额BALANCE1;

BALANCE1=BALANCE1+AMOUNT;

写回BALANCE1;

COMMIT;

}

- 两个更新操作要么全部完成要么全部不做否则就会使数据库处于不一致状态

- 若账户甲余额不足，程序可以发现并让事务滚回

- ◆ 撤销已作修改；恢复到正确状态

事务内部的故障

□什么是事务故障

- 某个事务在运行过程中由于种种原因未运行至正常终止点就夭折了

□事务内部的故障

- 有的是可以通过事务程序本身发现的
 - ◆ 转账事务的例子
- 有的是非预期的
 - ◆ 事务内部更多的故障是非预期的，是不能由应用程序处理的
 - ◆ 输入数据有误、运算溢出、违反了某些完整性限制、某些应用程序出错、并行事务发生死锁等

□以后，事务故障仅指这类非预期的故障

- 硬件故障使存储在外存中的数据部分丢失或全部丢失
 - 介质故障比前两类故障的可能性小得多，但破坏性大得多

- 介质故障的常见原因
 - 磁盘损坏、磁头碰撞
 - 操作系统的某种潜在错误
 - 瞬时强磁场干扰

8.2 数据库恢复的必要性

- 数据库系统可能发生各种各样的故障，大致可分为：
 - 系统故障
 - 事务内部的故障
 - 存储设备故障
 - 其它原因
 - ◆ 某些恶意的人为破坏，造成事务异常结束。如病毒，恶意流氓软件等
- 各类故障，对数据库的影响有两种可能性
 - 数据库本身被破坏
 - 数据库没有被破坏，但数据可能不正确
 - ◆ 由于事务的运行被非正常中止造成的

恢复的实现技术

□数据恢复的基本原理

- **数据冗余**：利用存储在系统其它地方的冗余数据来重建数据库中已被破坏或不正确的那部分数据

□恢复机制涉及的关键问题

1. 如何建立冗余数据

- ◆ 数据转储 (backup)
- ◆ 登记日志文件 (logging)

2. 如何利用这些冗余数据实施数据库恢复

□建立冗余数据最常用的技术是

- 数据转储和登记日志文件
- 通常在一个数据库系统中，这两种方法一起使用

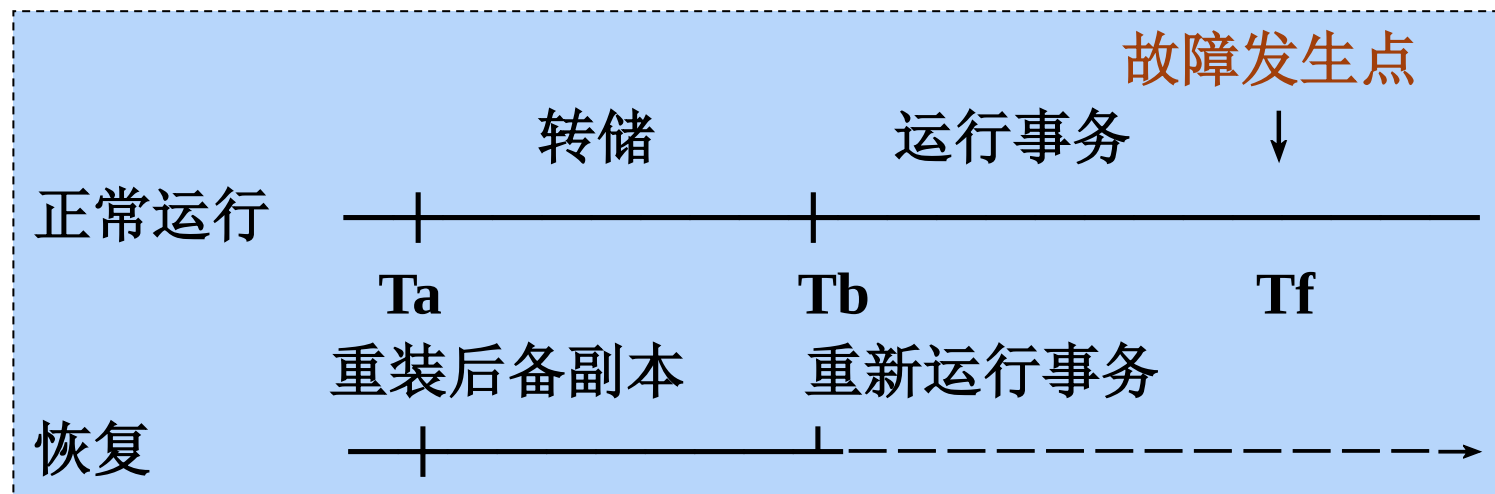
第8章 数据库恢复技术



- 8.1 事务的基本概念和特征
- 8.2 数据库恢复的必要性
- 8.4 数据转储与恢复
- 8.5 基于日志的数据库恢复
- 8.3 数据库恢复策略
- 8.6 检查点恢复技术
- 8.7 数据库镜像恢复技术
- 8.8 SQL Server的数据恢复机制
- 8.9 小结



- 数据转储是数据库恢复中采用的基本技术
 - DBA定期地将整个数据库复制到磁带或另一个磁盘上保存起来的过程
 - ◆ 这些备用的数据文本称为后备副本或后援副本
 - 数据库遭到破坏后可以将后备副本重新装入
 - ◆ 重装后备副本只能将数据库恢复到转储时的状态





□转储方法

1. 静态转储与动态转储
2. 海量转储与增量转储

		转储状态	
		动态转储	静态转储
转储方式	海量转储	动态海量转储	静态海量转储
	增量转储	动态增量转储	静态增量转储

1. 静态转储

- 在系统中无运行事务时进行转储
- 转储开始时数据库处于一致性状态
- 转储期间不允许对数据库的任何存取、修改活动
- 优点：实现简单
- 缺点：降低了数据库的可用性
 - 转储必须等用户事务结束
 - 新的事务必须等转储结束

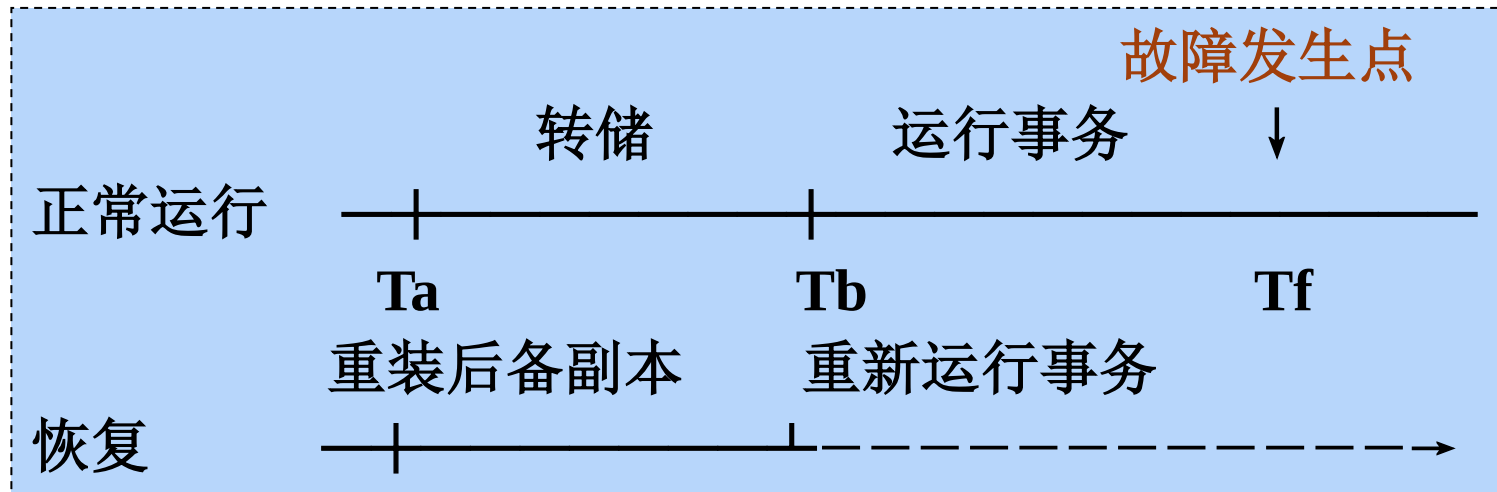
2. 动态转储

- 转储操作与用户事务并发进行
- 转储期间允许对数据库进行存取或修改
- 优点
 - 不用等待正在运行的用户事务结束
 - 不会影响新事务的运行

2. 动态转储

□ 缺点

- 不能保证副本中的数据正确有效，需要配合日志进行恢复
- [例] 在转储期间的某个时刻 T_c ，系统把数据 $A=100$ 转储到磁带上，而在下一时刻 T_d ，某一事务将 A 改为200。转储结束后，后备副本上的 A 过时



3. 海量转储与增量转储

- 海量转储：每次转储全部数据库
- 增量转储：只转储上次转储后更新过的数据
- 海量转储与增量转储比较
 - 从恢复角度看，使用海量转储得到的后备副本进行恢复往往更方便
 - 但如果数据库很大、事务处理十分频繁，则增量转储方式更实用更有效
- 应定期进行数据转储，制作后备副本
 - 但转储又是十分耗费时间和资源的，不能频繁进行
 - DBA应该根据数据库使用情况确定适当的转储周期和转储方法。例：每天晚上进行动态增量转储；每周进行一次动态海量转储；每月进行一次静态海量转储

第8章 数据库恢复技术



- 8.1 事务的基本概念和特征
- 8.2 数据库恢复的必要性
- 8.4 数据转储与恢复
- 8.5 基于日志的数据库恢复
- 8.3 数据库恢复策略
- 8.6 检查点恢复技术
- 8.7 数据库镜像恢复技术
- 8.8 SQL Server的数据恢复机制
- 8.9 小结

8.5 基于日志的数据库恢复

□ 8.5.1 数据库系统日志文件

- 记录有关事务的数据库操作信息
- 日志文件的格式
 - ◆ 以记录为单位的日志文件
 - ◆ 以数据块为单位的日志文件
- 日志文件内容
 - ◆ 事务标识
 - ◆ 操作类型
 - ◆ 操作对象
 - ◆ 更新前后的数据值



□以记录为单位的日志文件

- $\langle \text{Start } T_i \rangle$, 表示事务 T_i 开始
- $\langle T_i, X_j, V1, V2 \rangle$, 表示事务 T_i 对数据项 X_j 执行写操作。旧值是 $V1$, 新值是 $V2$
- $\langle \text{Commit } T_i \rangle$, 表示事务 T_i 已提交。事务对数据库所做的任何更新都写入到数据缓冲区中, 通常不能确定磁盘是否已经进行更新
- $\langle \text{Abort } T_i \rangle$, 表示事务 T_i 已中止。表明事务不能成功完成。如果事务中止, 系统将确保这一事务的更新不会对数据库造成影响

日志文件的作用

□事务故障恢复和系统故障恢复必须用日志文件

□介质故障恢复必须用日志文件

- 在动态转储方式中必须建立日志文件，后备副本与该日志文件综合起来才能将数据库恢复到一致性状态
- 与静态转储后备副本配合进行介质故障恢复
 - ◆ 静态转储的数据已是一致性的数据
 - ◆ 如果静态转储完成后，仍能定期转储日志文件，则在出现介质故障重装数据副本后，可以利用这些日志文件副本对已完成的事务进行重做处理
 - ◆ 可以把数据库恢复到故障前某一时刻的正确状态（不必重新运行那些已完成的事务程序）

8.5.1 数据库系统日志文件

- 为保证数据库是可恢复的，登记日志文件时必须遵循两条原则
 - 登记的次序严格按并行事务执行的时间次序
 - 必须先写日志文件，后写数据库
 - ◆ 写数据库和写日志文件是两个不同的操作
 - ◆ 在这两个操作之间可能发生故障，如果先写了数据库修改，而在日志文件中没有登记下这个修改，则以后就无法恢复这个修改了
 - ◆ 如果先写日志，但没有修改数据库，按日志文件恢复时只不过是多执行一次不必要的UNDO操作，并不会影响数据库的正确性

8.5.2 使用日志恢复数据库

□ 基于日志的恢复技术：

1. REDO技术

- 在日志中记录所有数据库写修改操作
- 如果发生故障，可以用REDO操作重做事务，恢复已完成的事务

8.5.2 使用日志恢复数据库

□ 基于日志的恢复技术：

2. UNDO恢复技术

- 在事务执行过程中修改了数据库而事务还没提交
- 此时如果系统崩溃，可以利用UNDO恢复技术（撤销事务），将被修改的数据项恢复到事务开始前的状态
- UNDO操作的过程
 - ◆ 检查日志文件，寻找事务 T_i 执行write(X)操作前写入日志的记录
 - ◆ 把数据库中的X项的值重新修改为更新前的旧值
 - ◆ 如果事务 T_i 有多个write操作，UNDO write操作的顺序必须与write操作时写入日志的顺序相反

第8章 数据库恢复技术

□8.1 事务的基本概念和特征

□8.2 数据库恢复的必要性

□8.4 数据转储与恢复

□8.5 基于日志的数据库恢复

□8.3 数据库恢复策略

- 8.3.1 事务故障的恢复
- 8.3.2 系统故障的恢复
- 8.3.3 介质故障的恢复

1. 事务故障的恢复

- 事务故障：事务在运行至正常终止点前被中止
- 恢复方法：利用日志文件撤销事务对数据的更改，系统回滚到事务执行前的状态
- 事务故障的恢复由系统自动完成，不需要用户干预
 - (1) 反向扫描文件日志，查找该事务的更新操作
 - (2) 对该事务的更新操作执行逆操作。即将日志记录中“更新前的值” (Before Image, BI) 写入数据库
 - (3) 继续反向扫描日志文件，查找该事务的其他更新操作，并做同样处理
 - (4) 如此处理下去，直至读到此事务的开始标记，事务故障恢复就完成了

2. 系统故障的恢复

- 系统故障造成数据库不一致状态的原因
 - 一些未完成事务对数据库的更新已写入数据库
 - 一些已提交事务对数据库的更新还留在缓冲区没来得及写入数据库
- 恢复方法
 - UNDO故障发生时未完成的事务
 - REDO已完成的事务
- 系统故障的恢复由系统在重新启动时自动完成, 不需要用户干预

系统故障的恢复步骤

- (1) 正向扫描日志文件（即从头扫描日志文件）
 - Redo队列：在故障发生前已经提交的事务
 - Undo队列：故障发生时尚未完成的事务
- (2) 对Undo队列事务进行UNDO处理
 - 反向扫描日志文件，对每个UNDO事务的更新操作执行逆操作
- (3) 对Redo队列事务进行REDO处理
 - 正向扫描日志文件，对每个REDO事务重新执行登记的操作

3. 介质故障的恢复

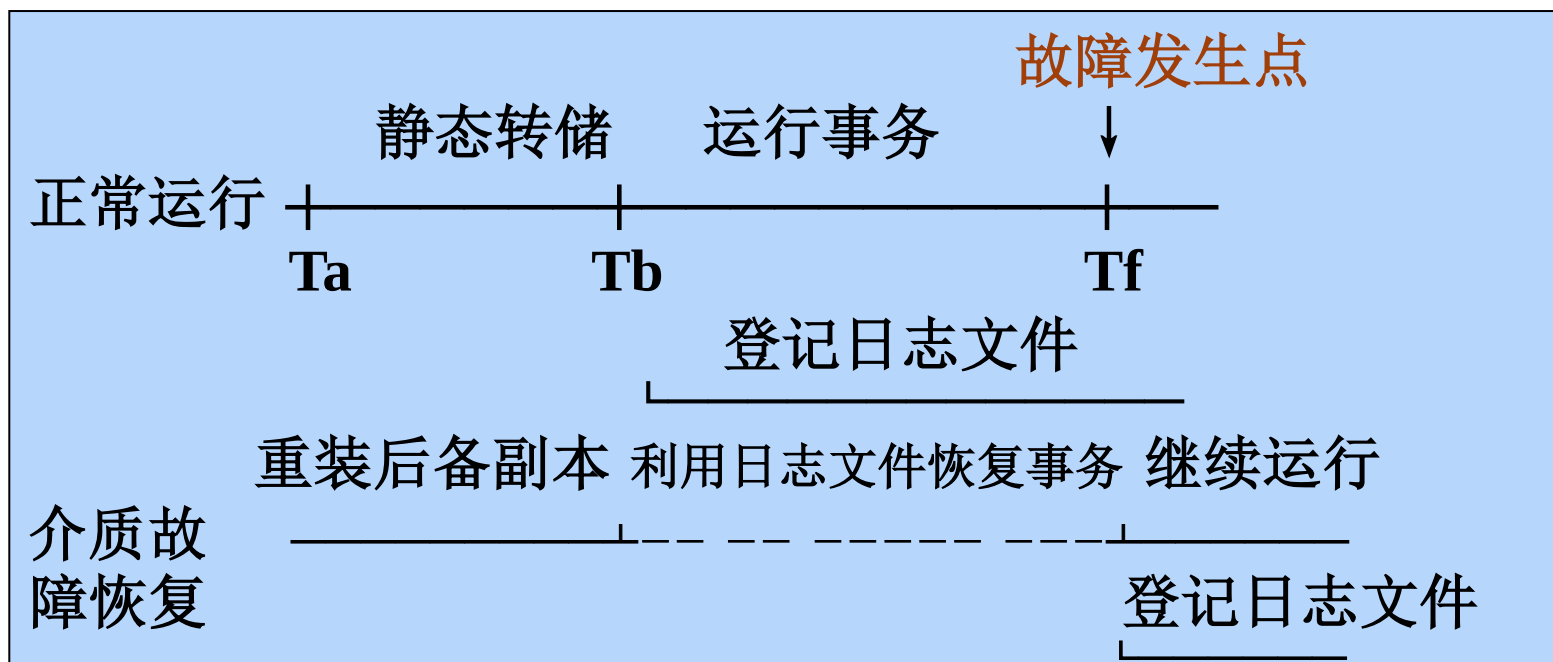
□恢复方法

- 重装数据库，使数据库恢复到一致性状态
- 重做已完成的事务

□介质故障的恢复需要DBA介入

- 重装最近转储的数据库副本和相关日志文件副本
- 执行系统提供的恢复命令
- 具体的恢复操作仍由DBMS完成

利用静态转储的副本进行故障恢复



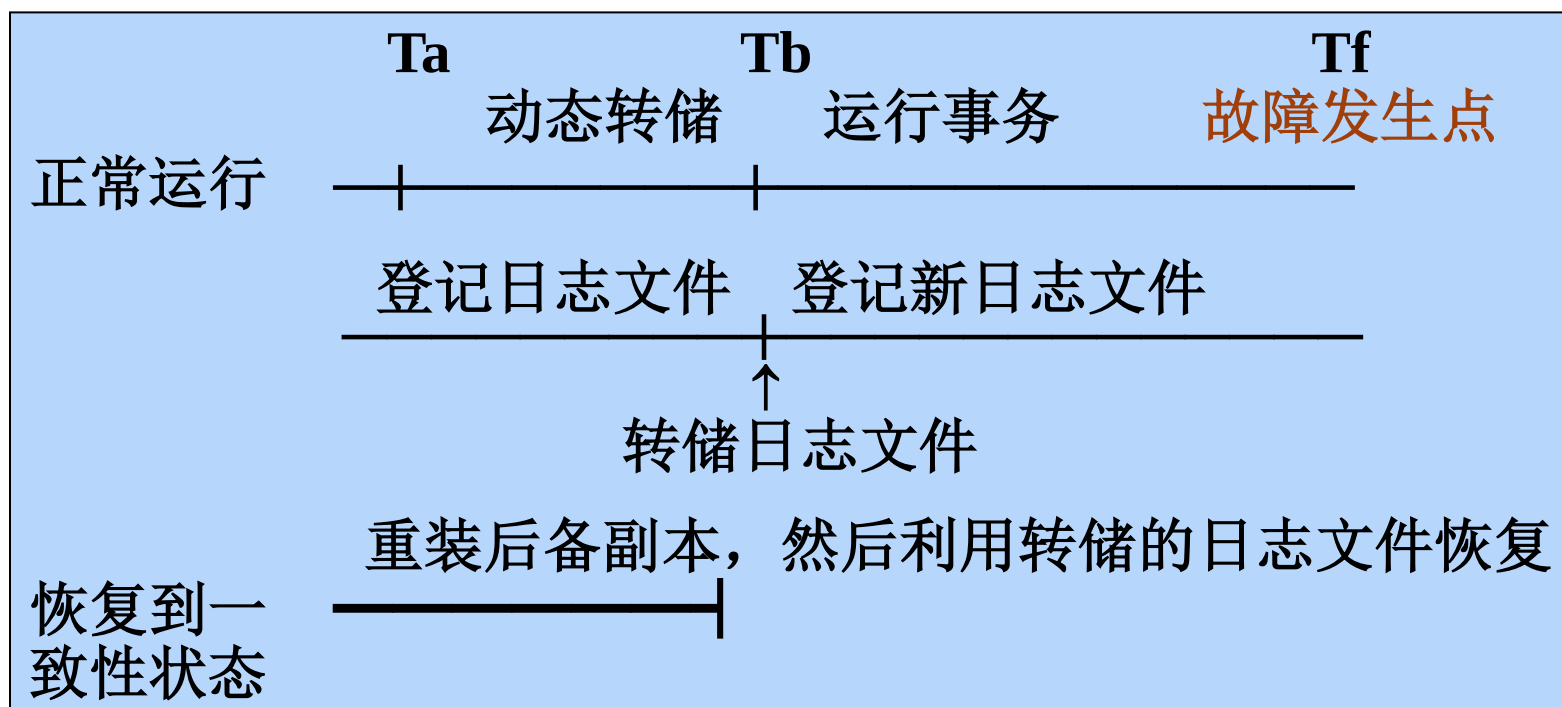
- ❑ 系统在 T_a 时刻停止运行事务，进行数据库转储
- ❑ 在 T_b 时刻转储完毕，得到 T_b 时刻的数据库一致性副本
- ❑ 系统运行到 T_f 时刻发生故障
- ❑ 为恢复数据库，首先由DBA重装数据库后备副本，将数据库恢复至 T_b 时刻的状态
- ❑ 可以利用这些日志文件副本对已完成的事务进行重做处理

利用动态转储的副本进行故障恢复



□ 利用动态转储得到的副本进行故障恢复

- 需要把动态转储期间各事务对数据库的修改活动登记下来，建立日志文件
- 后备副本加上日志文件才能把数据库恢复到某一时刻的正确状态



介质故障恢复步骤

(1) 装入最新的后备数据库副本，使数据库恢复到最近一次转储时的一致性状态

- 对于静态转储的数据库副本，装入后数据库即处于一致性状态
- 对于动态转储的数据库副本，还须同时装入转储时刻的日志文件副本，利用与恢复系统故障相同的方法（即 REDO+UNDO），才能将数据库恢复到一致性状态

(2) 装入有关的日志文件副本，重做已完成的事务

- 首先扫描日志文件，找出故障发生时已提交的事务的标识，将其记入重做队列
- 然后正向扫描日志文件，对重做队列中的所有事务进行重做处理。即将日志记录中“更新后的值”写入数据库

第8章 数据库恢复技术



- 8.1 事务的基本概念和特征
- 8.2 数据库恢复的必要性
- 8.4 数据转储与恢复
- 8.5 基于日志的数据库恢复
- 8.3 数据库恢复策略
- 8.6 检查点恢复技术
- 8.7 数据库镜像恢复技术
- 8.8 SQL Server的数据恢复机制
- 8.9 小结

8.6 检查点恢复技术

□问题的提出

- 利用日志技术进行数据库恢复时，恢复子系统必须搜索日志，确定哪些事务需要REDO，哪些事务需要UNDO，搜索整个日志将耗费大量的时间
- REDO处理：重新执行，浪费了大量时间

□为此发展了检查点 (Checkpoint) 技术，允许恢复子系统在登录日志期间动态维护日志：

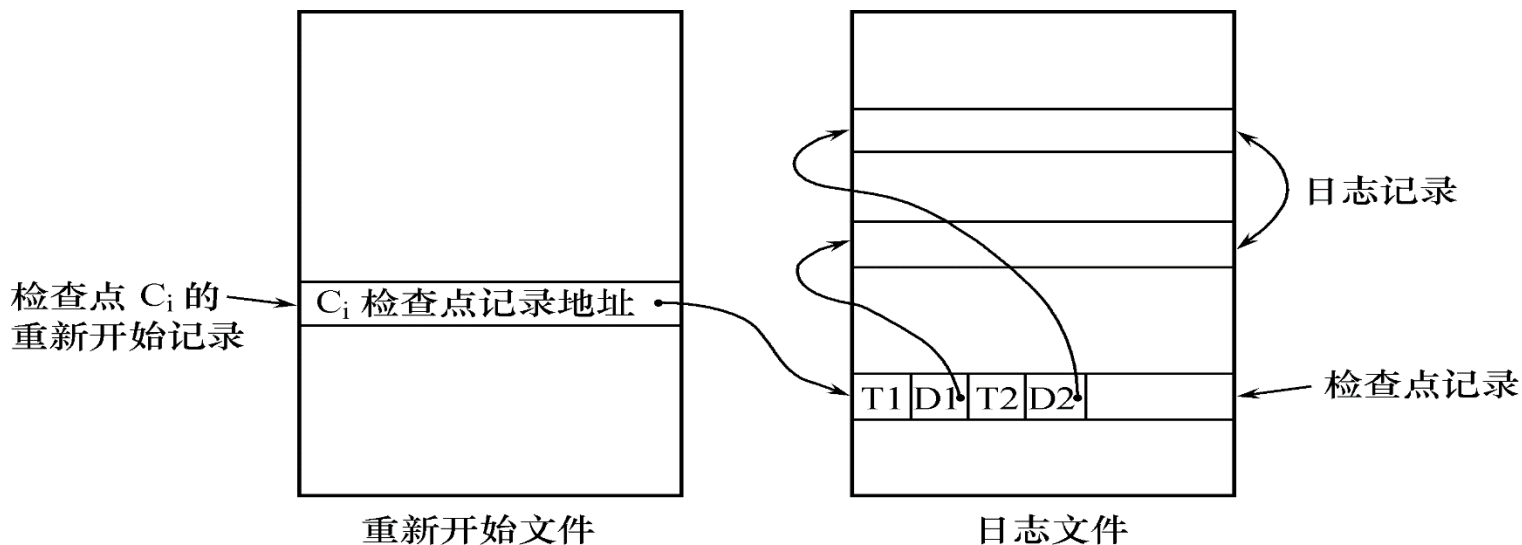
- 在日志文件中增加检查点记录
- 增加重启动文件（重新开始文件）

8.6 检查点恢复技术

- 检查点 (Checkpoint) 是记录在日志中表示数据库是否正在运行的一个标志，以记录所有当前活动的事务
- 检查点记录的内容
 - 1. 建立检查点时刻所有正在执行的事务清单
 - 2. 这些事务最近一个日志记录的地址
- 建立检查点原则
 - 定期：按照预定的一个时间间隔
 - 不定期：按照某种规则，如日志文件已写满一半建立一个检查点

8.6 检查点恢复技术

- **重启动文件**记录各个检查点记录在日志文件中的地址
- 周期性建立检查点，动态维护日志文件
 - 把日志缓冲区中的内容写入日志文件
 - 在日志文件中写入一个检查点记录
 - **把数据库缓冲区的内容写入数据库**
 - 把检查点记录在日志文件中的地址写入重启动文件



8.6 检查点恢复技术

□ 利用检查点的恢复步骤

- (1) 从重启动文件中找到最后一个检查点记录
- (2) 得到检查点时刻的事务清单；把事务清单中的事务列表暂时放入Undo队列
- (3) 从检查点开始正向扫描日志文件
 - ◆ 如有新开始的事务，暂时放入Undo队列
 - ◆ 如有提交事务 T_j ，把 T_j 从Undo队列移到Redo队列
- (4) 对Undo队列事务进行UNDO处理；对Redo队列事务进行REDO处理

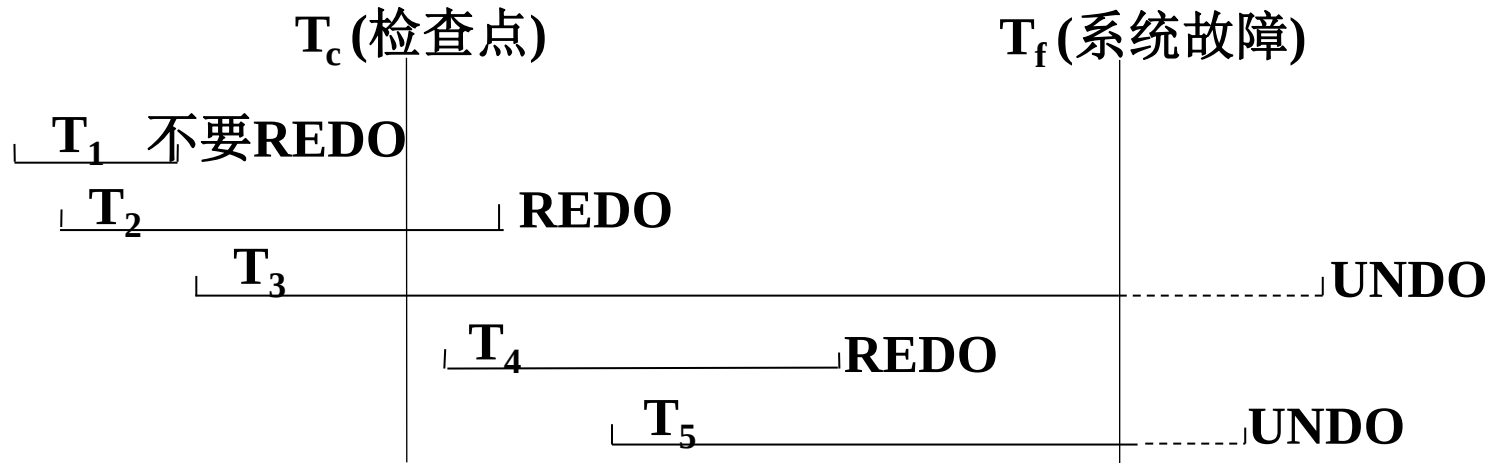
8.6 检查点恢复技术

□使用检查点方法可以改善恢复效率

- 当事务T在一个检查点之前提交
 - ◆ 由检查点记录知，T对数据库所做的修改已写入数据库，在进行恢复处理时，没有必要对事务T执行REDO操作
- 只运行检查点记录之后的操作即可

8.6 检查点恢复技术

□ 系统出现故障时，恢复子系统将根据事务的不同状态采取不同的恢复策略



恢复策略：

- T₃和T₅在故障发生时还未完成，所以予以撤销
- T₂和T₄在检查点之后才提交，它们对数据库所做的修改在故障发生时可能还在缓冲区中，尚未写入数据库，所以要REDO
- T₁在检查点之前已提交，所以不必执行REDO操作

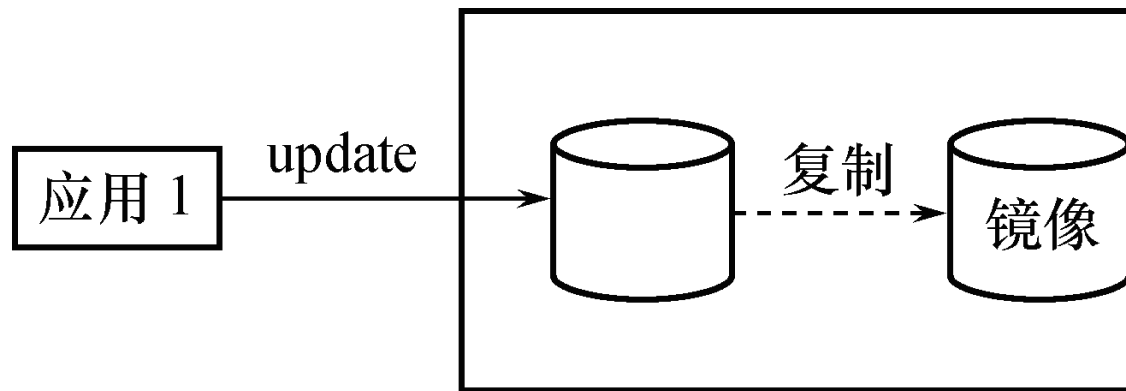
第8章 数据库恢复技术



- 8.1 事务的基本概念和特征
- 8.2 数据库恢复的必要性
- 8.4 数据转储与恢复
- 8.5 基于日志的数据库恢复
- 8.3 数据库恢复策略
- 8.6 检查点恢复技术
- 8.7 数据库镜像恢复技术
- 8.8 SQL Server的数据恢复机制
- 8.9 小结

8.7 数据库镜像恢复技术

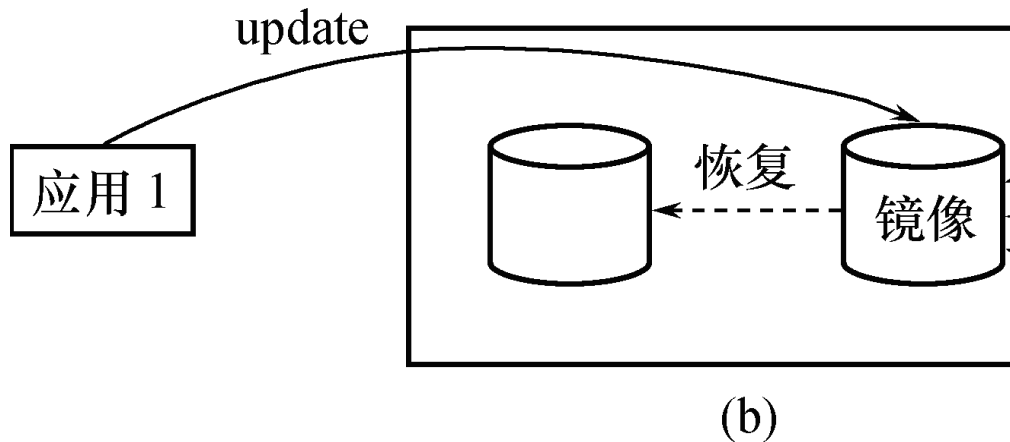
- 介质故障是对系统影响最为严重的一种故障
 - 介质故障恢复比较费时
 - 为预防介质故障，DBA必须周期性地转储数据库
- 提高数据库可用性的解决方案，数据库镜像
 - DBMS自动把整个数据库或其中的关键数据复制到另一个磁盘上
 - DBMS自动保证镜像数据与主数据的一致性
 - ◆ 每当主数据库更新时，DBMS自动把更新后的数据复制过去



8.7 数据库镜像恢复技术

□ 出现介质故障时

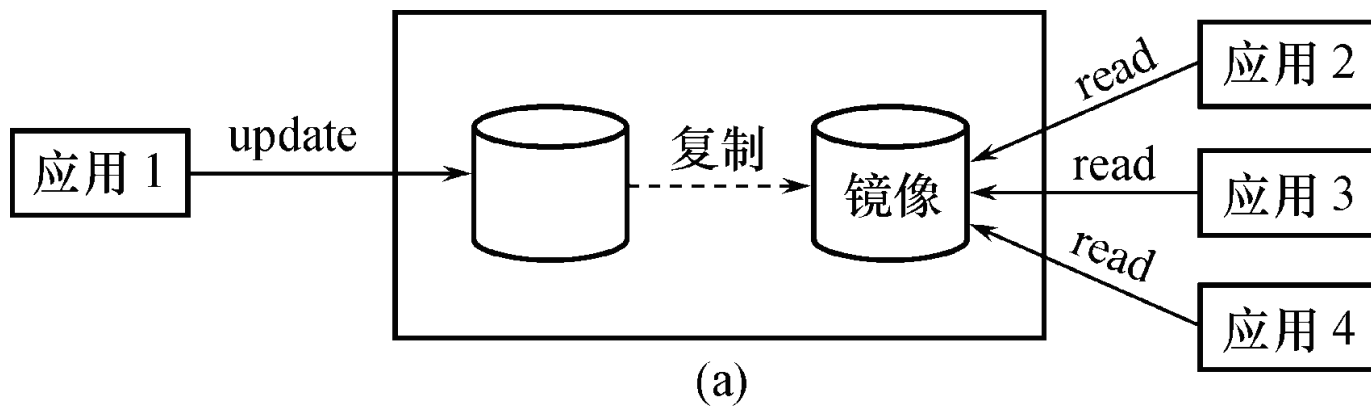
- 可由镜像磁盘继续提供使用
- 同时DBMS自动利用镜像磁盘数据进行数据库的恢复
- 不需要关闭系统和重装数据库副本



8.7 数据库镜像恢复技术

□ 没有出现故障时

- 可用于并发操作，即一个用户对数据加排它锁修改数据，其他用户可以读镜像数据库上的数据



- 频繁地复制数据自然会降低系统运行效率
- 在实际应用中用户往往只选择对关键数据和日志文件镜像，而不是对整个数据库进行镜像

第8章 数据库恢复技术



- 8.1 事务的基本概念和特征
- 8.2 数据库恢复的必要性
- 8.4 数据转储与恢复
- 8.5 基于日志的数据库恢复
- 8.3 数据库恢复策略
- 8.6 检查点恢复技术
- 8.7 数据库镜像恢复技术
- 8.8 SQL Server的数据恢复机制
- 8.9 小结

8.8 SQL Server的数据恢复机制



□ 应同时规划备份和还原过程

- 管理员必须首先确定数据库中的关键数据
- 必须确定这些数据是否可以将数据库还原到故障发生前一天晚上的状态，或者是否可以将数据库还原到尽可能接近故障发生的那一刻
- 还必须确定数据库在多长时间不可用，是否必须尽快使数据库重新联机，或者是否不需要立即还原

8.8 SQL Server的数据恢复机制



- 确定还原要求后，管理员就可以规划备份过程来维护满足还原要求的备份集
 - 管理员可以选择在运行时对系统的影响最小，同时又能满足还原要求的备份过程
 - 管理员还根据资源要求选择数据库的恢复模式
 - 恢复模式将针对完全恢复数据的重要程度来平衡记录开销

8.8 SQL Server的数据恢复机制



□ SQL Server提供的备份方法

- 数据库完全备份
- 数据库差异备份
- 事务日志备份
- 数据库文件或者文件组备份

1. 执行完全数据库备份

- 完整数据库备份，它备份包括事务日志的整个数据库
- 创建用于存放Northwind数据库完整备份的逻辑备份设备

```
USE master  
EXEC sp_addumpdevice 'disk', 'NwindBac',  
    'D:\MyBackupDir\NwindBac.bak'  
BACKUP DATABASE Northwind TO NwindBac
```



2. 执行差异备份

- 完全数据库备份虽然保证了数据的安全，但还原工作量太大。如果数据库频繁修改，那么用差异备份可以达到同样的效果，但还原所需时间少
- 执行差异备份特点
 - 用于频繁修改的数据库的情况下
 - 要求一个完全数据库备份
 - 备份上一次完全数据库备份后数据库中更改的部分
 - 节省备份和恢复过程的时间

2. 执行差异备份

- 在执行差异数据库备份时使用如下过程：
 - 创建定期的数据库备份
 - 在每个数据库备份之间定期创建差异数据库备份（例如，每隔四小时或四小时以上备份一次）
 - 如果使用完全恢复模型或大容量日志记录恢复模型，则创建事务日志备份的频率比差异数据库备份大，如每隔30分钟
 - 执行差异备份的示例

```
BACKUP DATABASE Northwind  
DISK = 'D:\MyData\MyDiffBackup.bak'  
WITH DIFFERENTIAL
```

3. 执行事务日志备份

- 事务日志是自上次备份事务日志后对数据库执行的所有事务的一系列记录。可以使用事务日志备份将数据库恢复到特定的即时点或恢复到故障点
- 一般情况下，事务日志备份比数据库备份使用的资源少
 - 可以比数据库备份更经常地创建事务日志备份。经常备份将减少丢失数据的危险
 - 事务日志备份有时比数据库备份大。例如，数据库的失误率很高，从而导致事务日志迅速增大。在这种情况下，应更经常地创建事务日志备份

3. 执行事务日志备份

- 事务日志备份只能与完全恢复模型和大容量日志记录恢复模型一起使用
- 备份从上一次成功执行BACKUP LOG语句之后到当前事务日志结尾的这段事务日志

```
USE master
EXEC sp_addumpdevice 'disk', 'NwindBacLog',
    'D:\Backup\NwindBacLog.bak'
BACKUP LOG Northwind TO NwindBacLog
```

4. 执行数据库文件或者文件组备份



- 在实际生活中有一些超大型数据库，对它们进行完全数据库备份可能无法管理。可以只备份一部分文件
- 文件备份操作必须与事务日志备份一起使用。因此，文件备份只适用于完全恢复模型和大容量日志记录恢复模型
- 还原文件后，必须还原自创建文件备份后创建的事务日志备份以使数据库处于一致状态
- 与数据库备份相比，文件备份的主要缺点是增加了管理的复杂性
 - 必须注意维护完整的文件备份集和覆盖的日志备份

4. 执行数据库文件或者文件组备份



□ 执行数据库文件或者文件组备份有以下优点：

- 能够更快地恢复已隔离的媒体故障，可以迅速还原损坏的文件
- 可以同时创建文件和事务日志备份，使我们得以维护定期日志备份调度
- 文件备份在调度和媒体处理上具有更大的灵活性，如超大数据库备份

□ 执行数据库文件或者文件组备份的特点

- 用于大型数据库
- 备份单个数据库文件

4. 执行数据库文件或者文件组备份



□ 执行数据库文件或者文件组备份时:

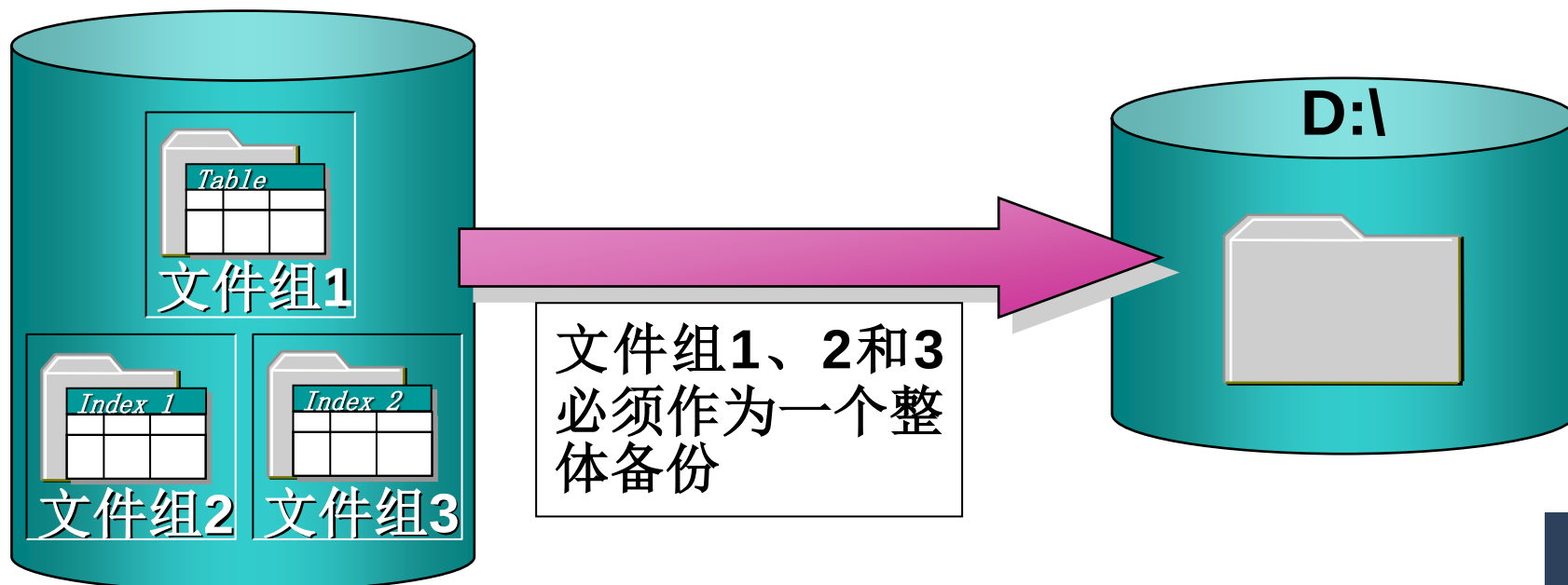
- 必须指定逻辑文件或者文件组
- 为了使还原的文件与数据库的其他部分相一致，必须执行事务日志备份
- 为了确保定期备份所有的数据库文件或者文件组，应该制定轮流备份每个文件的计划
- 最多可以指定16个文件或者文件组

```
BACKUP DATABASE Phoneorders  
FILE = Orders2 TO OrderBackup2  
BACKUP LOG PhoneOrders to OrderLog
```

4. 执行数据库文件或者文件组备份



- 如果创建了索引，必须将几个数据库文件作为一个整体备份
 - 如果在一个文件组里创建了索引和表，必须将整个文件组作为一个整体备份
 - 如果索引创建在多个文件组中，而表创建在另一个文件组中，必须将所有文件组作为一个整体备份



8.8 SQL Server的数据恢复机制



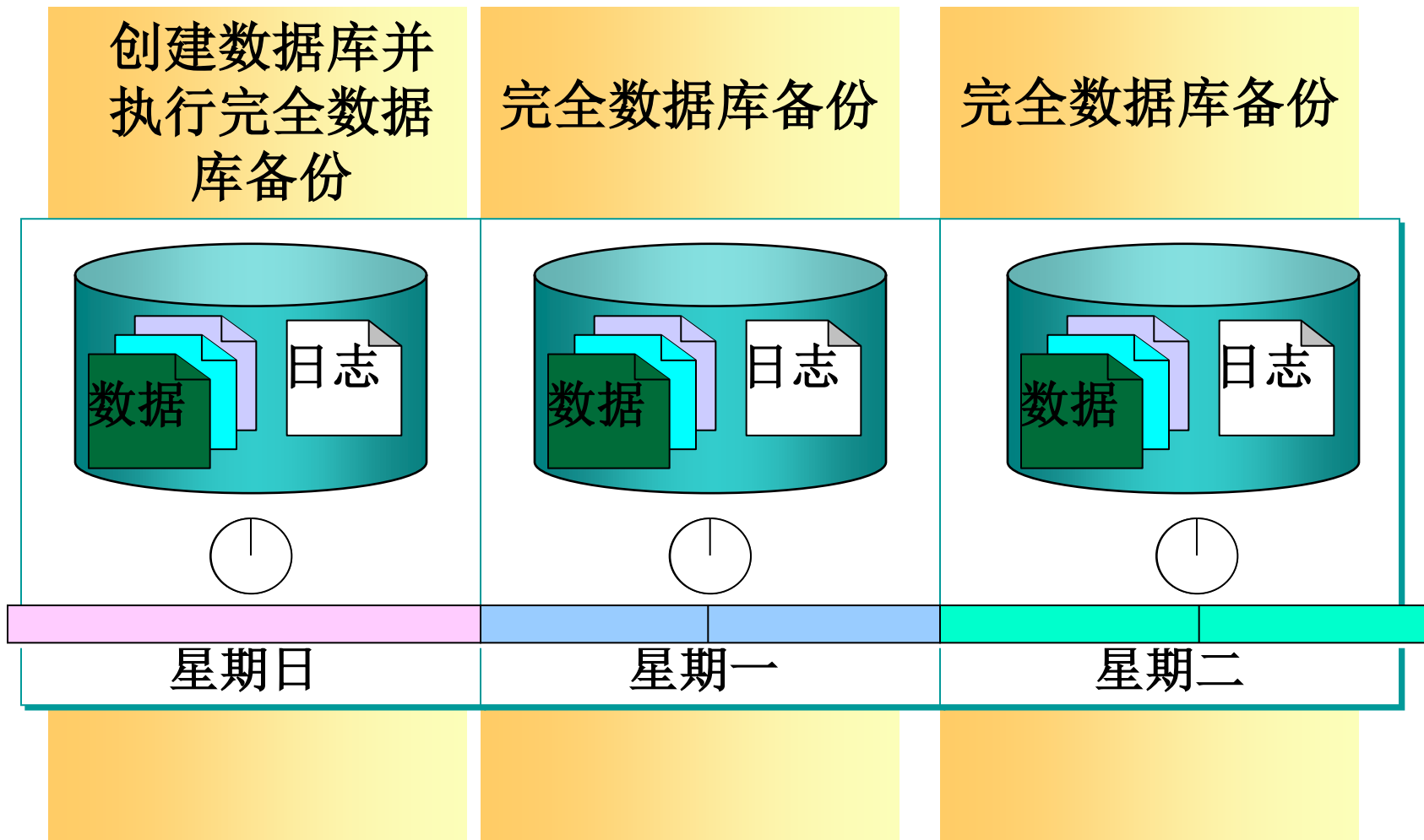
□ SQL Server提供的备份方法

- 数据库完全备份
- 数据库差异备份
- 事务日志备份
- 数据库文件或者文件组备份

□ SQL Server中四种数据库备份策略

- 1. 完全数据库备份策略
- 2. 完全数据库和事务日志备份策略
- 3. 差异备份策略
- 4. 数据库文件或文件组备份策略

1. 完全数据库备份策略



注意：在执行完全数据库备份策略后应清理事务日志

1. 完全数据库备份策略

- 完全备份就是通过海量转储形成的备份
 - 其最大优点是恢复数据库的操作简便，它只需要将最近一次的备份恢复
- 完全备份所占的存储空间很大且备份的时间较长，只能在一个较长的时间间隔上进行完全备份
- 其缺点是当根据最近的完全备份进行数据恢复时，完全备份之后对数据所做的任何修改都将无法恢复
- 当数据库较小、数据不是很重要或数据操作频率较低时，可采用完全备份的策略进行数据备份和恢复

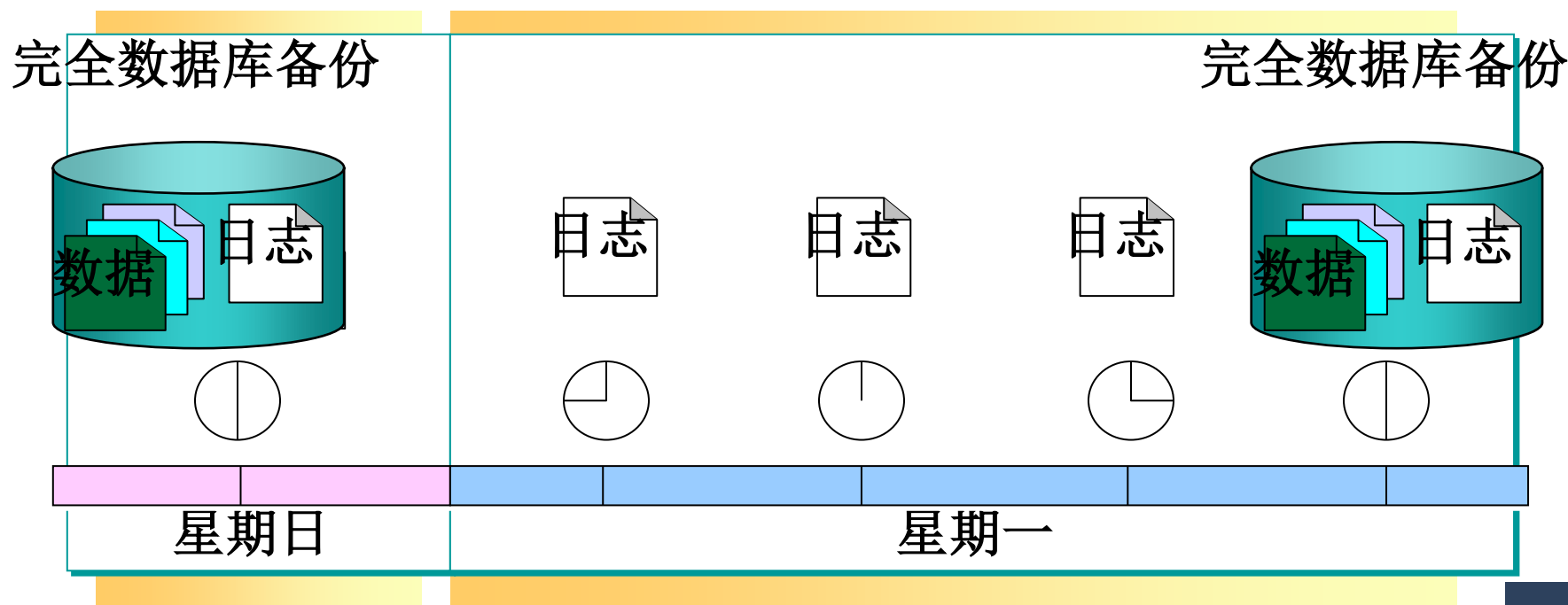
2. 完全数据库和事务日志备份策略



□ 由于完全数据库备份不方便频繁执行

- 比如一个公司的数据库每周一进行一次备份，那么如果周三系统崩溃，两天的操作就没有了

□ 可以采用完全数据库和事务日志备份结合的方法



2. 完全数据库和事务日志备份策略

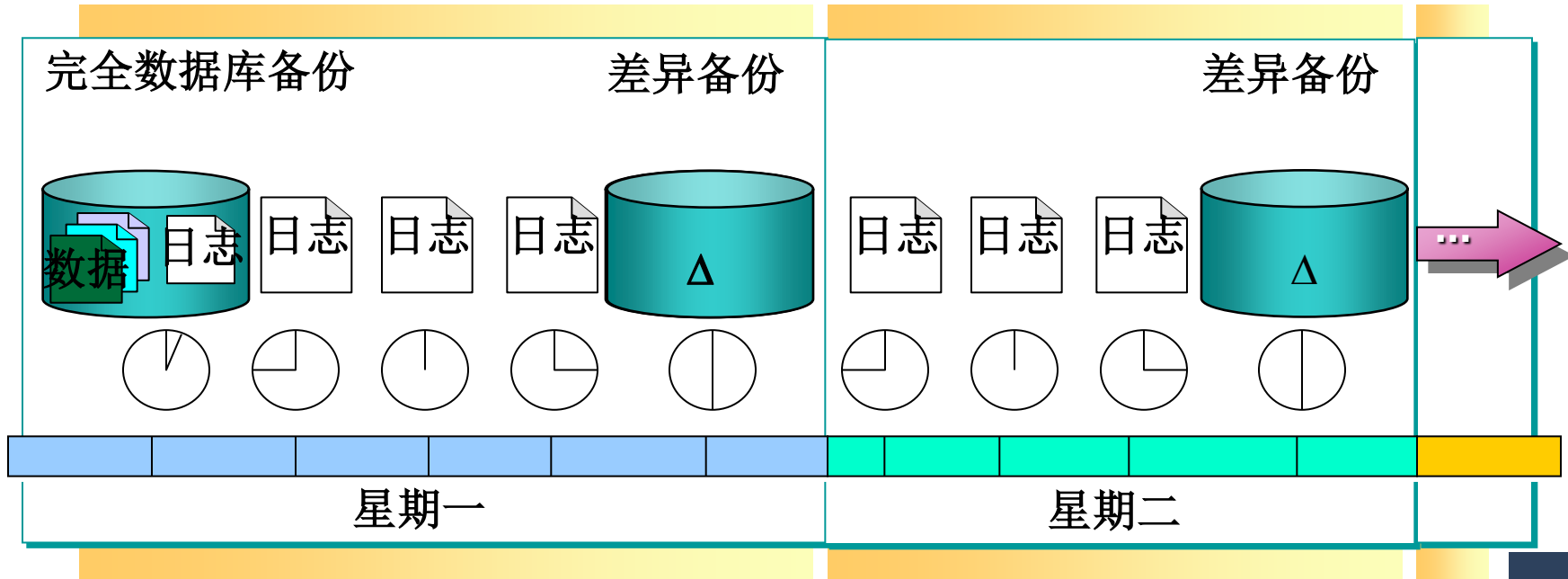


- 事务日志备份必须与数据库的完全备份联合使用，才能实现数据备份和恢复功能
 - (1) 定期进行完全备份，例如一天一次或两天一次
 - (2) 更频繁地进行事务日志备份，如一小时一次或两小时一次等
- 当需要数据库恢复时，首先用最近一次完全备份恢复数据库，然后用最近一次完全备份之后创建的所有事务日志备份，按顺序恢复完全备份之后发生在数据库上的所有操作
- 完全备份和事务日志备份相结合的方法，能够完成许多数据库的恢复工作。但对那些不在事务日志中留下记录的操作，仍无法恢复数据

3. 差异备份策略

差异数据库备份可以节省事务日志的恢复时间

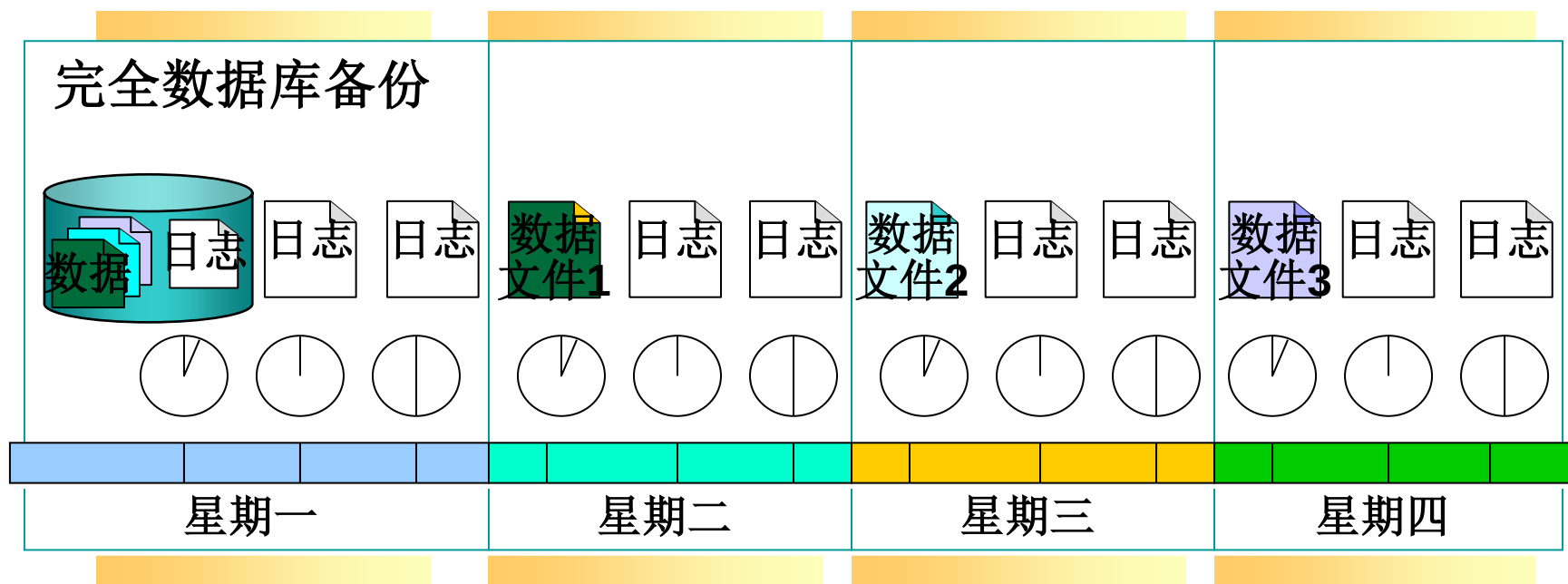
- 实施差异备份策略时，必须包含完全数据库备份和事务日志备份
- 使用数据库备份、差异数据库备份和事务日志备份的组合，可以将故障恢复所需的时间减到最少



4. 数据库文件或文件组备份策略



- 在超大型数据库的备份中，由于不能用完全数据库备份，只能用数据库文件或文件组备份策略
- 实施数据库文件或文件组备份策略时必须包含完全数据库备份和事务日志备份



8.8 SQL Server的数据恢复机制



- 可以为SQL Server中的每个数据库选择三种恢复模型中的一种，以确定如何备份数据以及能承受何种程度的数据丢失
 - 简单恢复
 - ◆ 简单恢复允许将数据库恢复到最新的备份
 - 完全恢复
 - ◆ 完全恢复允许将数据库恢复到故障点状态
 - 大容量日志记录恢复
 - ◆ 大容量日志记录恢复允许大容量日志记录操作

三种恢复模型的优点和含义



恢复模型	优点	工作损失表现	能否恢复到即时点?
简单	允许高性能大容量复制操作。 收回日志空间以使空间要求最小。	必须重做自最新的数据库或差异备份后所发生的更改。	可以恢复到任何备份的结尾处。随后必须重做更改。
完全	数据文件丢失或损坏不会导致工作损失。 可以恢复到任意即时点（例如，应用程序或用户错误之前）。	正常情况下没有。 如果日志损坏，则必须重做自最新的日志备份后所发生的更改。	可以恢复到任何即时点。
大容量日志记录	允许高性能大容量复制操作。 大容量操作使用最少的日志空间。	如果日志损坏，或者自最新的日志备份后发生了大容量操作，则必须重做自上次备份后所做的更改。 否则不丢失任何工作。	可以恢复到任何备份的结尾处。随后必须重做更改。

8.8 SQL Server的数据恢复机制



- 还原数据库应根据实际情况从不同的备份中还原
 - 1. 从完全数据库备份还原
 - 2. 从差异备份还原
 - 3. 还原事务日志备份
 - 4. 从文件或文件组还原

1. 从完全数据库备份还原

□何时使用

- 物理磁盘受损
- 整个数据库毁坏、受损或被删除
- 在另一个SQL Server保留相同的数据库拷贝

□指定恢复选项

- 用RECOVERY选项启动恢复过程
- 用NORECOVERY选项延迟恢复过程

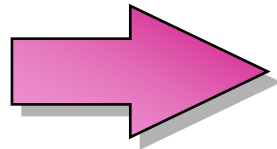
```
USE master  
RESTORE DATABASE Northwind  
FROM NwindBac  
WITH FILE = 2, RECOVERY
```


2. 从差异备份还原

- 所谓差异还原，就是只还原数据库中自上一次数据库备份之后修改过的所有的页，即只还原数据库中自最近的完全数据库备份以来所更改的部分
- 使数据库恢复到执行差异备份时的情形
- 还原差异备份比应用一系列表示相同活动的事务日志更快

语法与完全数据库还原相同

指定包含差异备份的备份文件



```
USE master
RESTORE DATABASE Northwind
FROM NwindBacDiff
WITH NORECOVERY
```

3. 还原事务日志备份

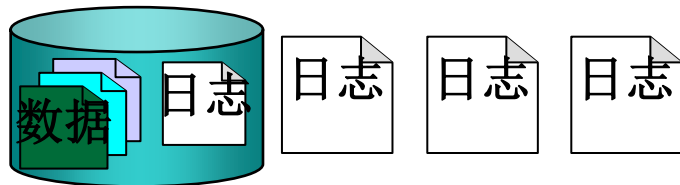
□恢复事务日志是非常有必要的

- SQL Server使用各数据库的事务日志来恢复事务
- 事务日志是数据库中已发生的所有修改和执行每次修改的事务的一连串记录
- 事务日志记录每个事务的开始。它记录了在每个事务期间，对数据的更改及撤销所做更改（以后如有必要）所需的足够信息
- 对于一些大的操作（如CREATE INDEX），事务日志则记录该操作发生的事实

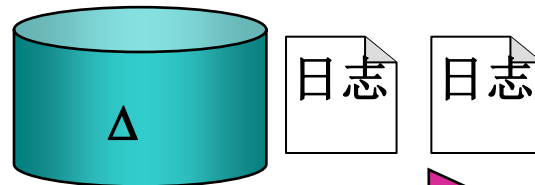
3. 还原事务日志备份

Northwind数据库备份

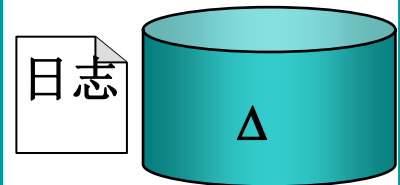
完全数据库



差异



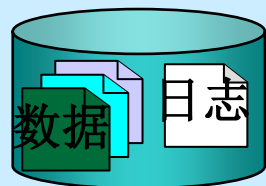
差异



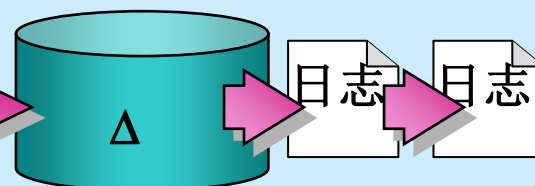
数据库损坏

还原Northwind数据库

完全数据库



差异



```
USE master
RESTORE LOG Northwind
FROM NwindBacLog
WITH FILE = 2, RECOVERY
```

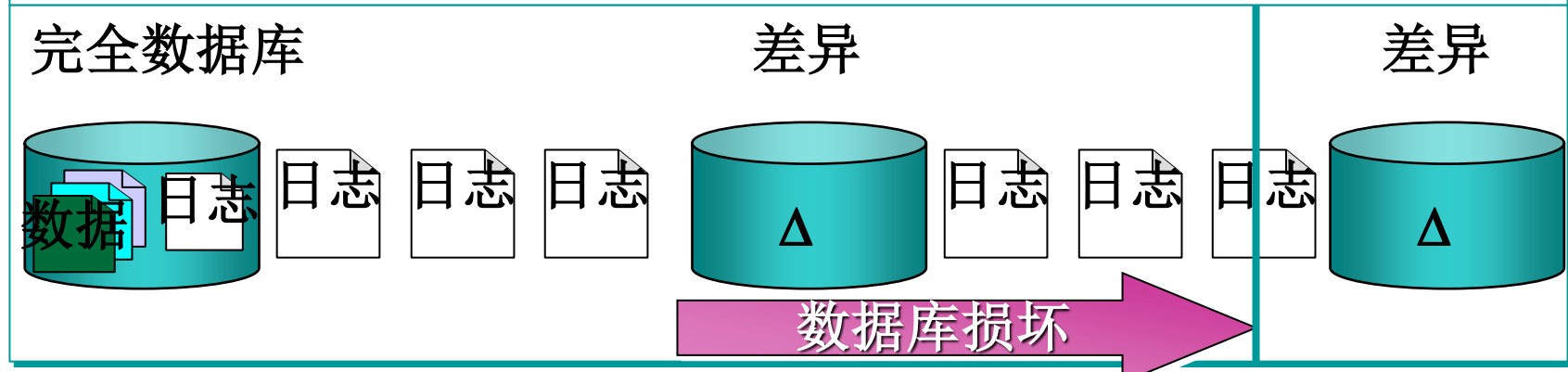
3. 还原事务日志备份

□还原到指定时间点的方法：

- 执行使用NORECOVERY子句的RESTORE DATABASE语句
- 执行RESTORE LOG语句以应用每个事务日志备份，同时指定：
 - ◆ 事务日志将应用到的数据库的名称
 - ◆ 要从其中还原事务日志备份的备份设备
 - ◆ RECOVERY和STOPAT子句。如果事务日志备份不包含要求的时间（例如，如果指定的时间超出了事务日志所包含的时间范围），则会生成警告，并且数据库将保持未恢复的状态

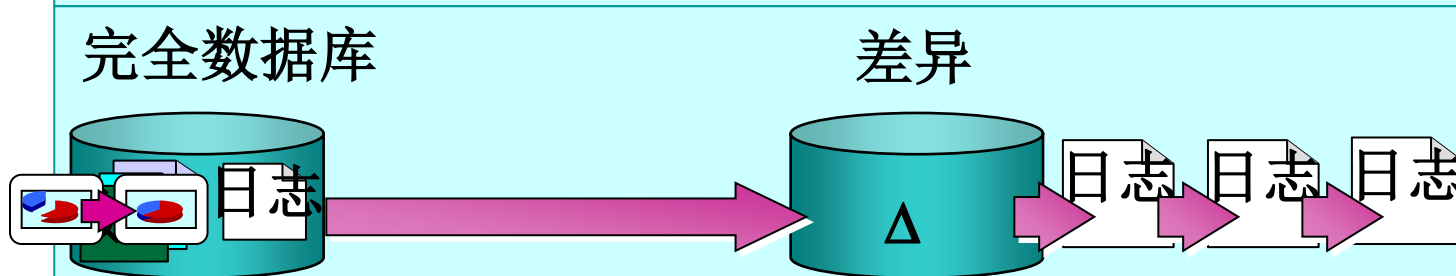
指定时间点

Northwind数据库备份



```
USE master
RESTORE LOG Northwind
FROM NwindBacLog
WITH FILE = 2, RECOVERY,
STOPAT = 'January 3, 2000 1:00 AM'
```

还原Northwind数据库



4. 从文件或文件组还原

□ 文件组允许对文件进行分组

- 例如，可以分别在三个硬盘驱动器上创建三个文件 (Data1.ndf、Data2.ndf和Data3.ndf)，并将这三个文件指派到文件组fgroup1中
- 然后，可以明确地在文件组fgroup1上创建一个表。对表中数据的查询将分散到三个磁盘上，因而性能得以提高

□ 如果有文件或文件组备份，还原的方法

- 应用所有文件备份后创建的事务日志
- 把包含索引和表的文件组备份作为一个整体还原

4. 从文件或文件组还原

□如何还原文件和文件组：

(1) 执行RESTORE DATABASE语句以还原文件和文件组备份，同时指定：

- 要还原的数据库名称
- 要从其中还原数据库备份的备份设备
- 对于每个要还原的文件，指定FILE子句
- 对于每个要还原的文件组，指定FILEGROUP子句
- NORECOVERY子句。如果在创建文件备份之后没有对文件进行修改，则指定RECOVERY子句

4. 从文件或文件组还原

(2) 若在创建文件备份之后对文件进行了修改，则执行 RESTORE LOG 语句以应用事务日志备份，同时指定：

- 事务日志将应用到的数据库的名称
- 要从其中还原事务日志备份的备份设备
- NORECOVERY 子句，前提是在当前事务日志备份之后，还要应用其它事务日志备份，否则应指定 RECOVERY 子句。事务日志备份（如果可用）必须包括截止到日志结尾处的文件和文件组备份（除非已还原 ALL 数据库文件）

```
USE master
RESTORE DATABASE Northwind
FILE = Nwind2
FROM Nwind2Bac WITH NORECOVERY
```


8.9 小结

- 如果数据库只包含成功事务提交的结果，就说数据库处于一致性状态。保证数据一致性是对数据库的最基本的要求
- 事务是数据库的逻辑工作单位，DBMS保证系统中一切事务的原子性、一致性、隔离性和持续性
 - 事务概念及其特性
 - 显示事务的定义方法
- DBMS必须对事务故障、系统故障和介质故障进行恢复
 - 故障的种类及其恢复策略
 - 恢复中最经常使用的技术：
 - ◆ 数据库转储：故障恢复过程、数据转储分类
 - ◆ 登记日志文件：日志作用、工作原理

8.9 小结

□恢复的基本原理

- 利用存储在后备副本、日志文件和数据库镜像中的冗余数据来重建数据库

□常用恢复技术

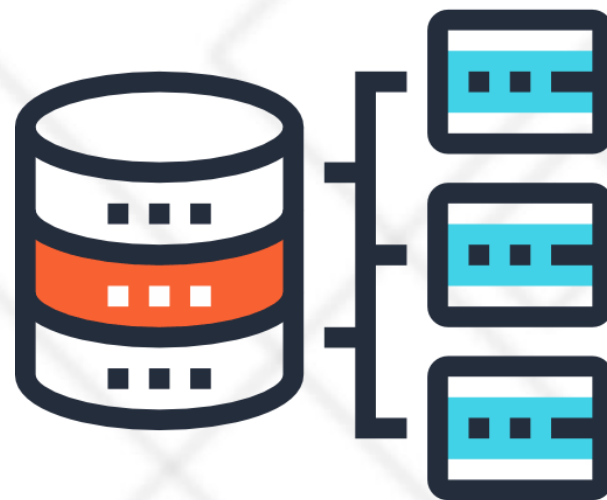
- 事务故障的恢复 UNDO
- 系统故障的恢复 UNDO + REDO
- 介质故障的恢复：
 - ◆ 重装备份并恢复到一致性状态 + REDO

□提高恢复效率的技术

- 检查点技术：可以提高系统故障的恢复效率；可以在一定程度上提高利用动态转储备份进行介质故障恢复的效率
- 镜像技术：可以改善介质故障的恢复效率



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY



作业

《数据库系统原理》第8章

习题1, 3, 4, 8